Lab04. Storing containers in the registry

Creation and Administration of Cloud-based Applications

Wojciech Thomas

Fall 2026

1 Learning objectives

1. Learn what is container registry.
2. Use the ECR (Elastic Container Registry) to keep built container images.

2 Exercises

Container images are stored in a service named **registry**.

When you install Docker, it always creates **local registry**.

You can install your own registry in the remote machine, or you can use one of many hosted solutions (e.g. DockerHub, AWS Elastic Container Registry, etc.). Each registry can be public or private. You can always download container images from **public registry**. To download container images from **private registry** you must log in first. If you want to upload your own image to any remote registry, you always have to log in.

If you want to run a container, and the container image is not present in the local registry, image is (by default) automatically downloaded from the DockerHub.

2.1 Start

1. Start your AWS Learning Lab, and open AWS Console.
2. Start development environment created in the previous lab.

2.2 Working with ECR

1. In the new tab open AWS console and select Elastic Container Registry service.
2. Click button Get Started to create a new registry.
3. Set the following settings:
   - Visibility: Private
   - Repository name: flask-app
4. Click Create repository button.
5. Open `flask-app` registry. Click View push commands button to learn how to push images to registry.

6. Open CACBA environment in the browser. Make sure that you have a repository with your Flask project available. If not – clone it from CodeCommit.
7. Build a new container image using Dockerfile in your project, and then verify if it is present in **local** container registry:

   ```
   docker build -t flask-app .
   docker images
   ```

8. Log in to the remote registry:

   ```
   aws ecr get-login-password --region us-east-1 | docker login --username AWS
   ↪  --password-stdin <ECR-DNS>
   ```

   Remember to replace registry DNS address with your actual registry address! `<ECR-DNS>` should look like `051548163354.dkr.ecr.us-east-1.amazonaws.com`

9. Add an additional tag your image, before pushing it to the ECR:

   ```
   docker tag flask-app:latest <ECR-DNS>/flask-app:latest
   docker images
   ```

   Replace `<ECR-DNS>` repository name, with your repository DNS address!

   Compare the list of images in local registry with the previous step.

10. Push your container image from local registry to the ECR registry:

    ```
    docker push <ECR-DNS>/flask-app:latest
    ```

11. Open the ECR service in the AWS console, and verify if the current version of the image is stored in the remote registry.
12. In the terminal display the list of container images:

    ```
    docker images
    ```

    Remove images created in this lab (`flask-app`). As these images are referenced by remote repository you have to force the deletion:

    ```
    docker rmi <IMAGE-ID> -f
    ```

    Replace `<IMAGE-ID>` with the ID of your image. You don't have to copy whole number, usually 4 hex digits are enough.

13. Verify that the image was actually deleted from local repository.

    At this moment, you have no image in your local registry, but the image is still present in the remote ECR registry.

14. Pull image from the remote repository into your local repository:

    ```
    docker pull <ECR-DNS>/flask-app:latest
    ```

15. Display the list of local images and verify if the image was actually pulled.
16. Start container from pulled image. Replace the container label with your real name! :

    ```
    docker run -i -t -p 5000:5000 <ECR-DNS>/flask-app
    ```

    The container is started in interactive (`-i`) mode, so you can observe output from the application.

17. Open new terminal and verify if container with Python application really works:

    ```
    curl http://localhost:5000/
    ```

18. Display running containers: `docker ps`

19. Stop the container: `docker stop <CONTAINER-ID>`
20. Show all containers (including stopped): `docker ps -a`
21. Start the stopped container: `docker start <CONTAINER-ID>` and verify if application is really working.
22. Stop the running container.
23. Remove the container from the memory: `docker rm <CONTAINER-ID>`.

3 Read more

1. Read more on Docker doc pages: What is a registry?.
2. Visit DockerHub registry and review images available.